

Combat System

Overview

I designed and implemented a **data-driven combat system** for a 5-person isometric space-horror ARPG in Unity. The system was built to be flexible for designers while providing responsive and readable combat feedback.

Core Systems

Weapon & Attack System

- Data-driven weapon configuration using Unity ScriptableObjects
- Supports ranged, melee and unarmed attacks
- Semi-automatic and automatic weapons
- Ammo, reloads and fire-rate cooldowns
- Hitscan raycasts and melee sphere casts
- Distance-based damage falloff

Aiming & Combat Feedback

- Mouse-based aiming with player rotation
- Dynamic crosshair feedback based on hit probability
- Crosshair color communicates accuracy
- Increasing crosshair jitter communicates weapon inaccuracy
- Cinemachine aim offset for improved aiming feel

Hit Chance System

A custom hit-chance model evaluates the player's combat state using factors such as:

- Distance to target
- Health and stamina
- Movement and sprinting
- Weapon-specific accuracy modifiers
- Weapon damage/range curves

The resulting score drives both **visual feedback and ballistic accuracy**, allowing the player to understand shot quality without relying on first-person aiming mechanics.

Ballistics

For weapon spread, I implemented Gaussian-distributed bullet deviation using the **Box-Muller transform**. This produces a more natural clustered spray around the intended aim direction compared to uniform random spread.

Architecture

The combat system is split into focused components:

AttackInput → PlayerAimController → PlayerWeaponHandler → PerformAttack → ImpactProcessor

Weapon behaviour and tuning are defined through ScriptableObjects, keeping gameplay logic separate from weapon data and making the system easier to extend.

My Contribution

I was primarily responsible for the **combat system**, including weapon handling, attack execution, hit chance, ballistic spread and combat feedback. I also contributed to **player audio and animations**.

Technical Highlights

- C# / Unity
- Data-driven architecture
- ScriptableObjects
- Event-driven input
- Hitscan & physics-based melee
- Gaussian/Box-Muller ballistic distribution
- Custom hit-chance algorithm
- Cinemachine integration
- Designer-friendly weapon configuration

My very proud implementation of the box-muller transform

```
public static class BallisticsUtility
{
    public static Vector3 GetGaussianSpread(Vector3 forward, float spreadStandardDeviation)
    {
        var u1 = Random.value;
        var u2 = Random.value;

        // Box-Muller transform to get two independent standard normals
        var offsetX = Mathf.Sqrt(-2f * Mathf.Log(u1)) * Mathf.Cos(2f * Mathf.PI * u2);
        var offsetY = Mathf.Sqrt(-2f * Mathf.Log(u1)) * Mathf.Sin(2f * Mathf.PI * u2);

        // Scale by the weapon's spread standard deviation
        offsetX *= spreadStandardDeviation;
        offsetY *= spreadStandardDeviation;

        // The spread offsets will be applied along these axes
        var right = Vector3.Cross(Vector3.up, forward.normalized);
        var up = Vector3.Cross(forward, right).normalized;

        // Return the new direction with spread applied
        return (forward + offsetX * right + offsetY * up).normalized;
    }
}
```

